

TDDE01 lab3 report

Zurab Gergaia, Oskar Andersson, Ahmad Soltani

December 2025

Statement of Contribution

- Ahmad Soltani did all the work for part 2, **Kernel Methods**, and answered question 1.1.
- Oskar Andersson did all the work for part 3, **Support Vector Machines**, and answered question 1.2.
- Zurab Gergaia did all the work for part 4, **Neural Networks**, and answered question 1.3.

1 Theory

1.1 What is the kernel trick?

The kernel trick (page 231) is a method applied to Bayesian linear regression to transform it into a Gaussian process construction. Instead of explicitly managing these weights and features, the trick allows the model to be defined essentially by a kernel acting as the covariance function $\kappa(x, x')$. This substitution leads to a formulation where the mean function is zero and establishes the theoretical connection between Bayesian linear regression, kernel ridge regression, and Gaussian processes.

1.2 In the literature, it is common to see a formulation of SVMs that makes use of a hyperparameter C . What is the purpose of this hyperparameter?

On page 210/211, the book refers to a hyperparameter C which is used for regularization. It is typically defined as either $\frac{1}{2\lambda}$ or $\frac{1}{2n\lambda}$, where n is the size of the training set and λ is a regularization parameter.

1.3 In neural networks, what do we mean by mini-batch and epoch?

According to the book (page 125) **mini-batch** is a small subsample of the training data used to compute an approximation of the gradient during one iteration of the optimization algorithm (stochastic gradient descent). While utilizing mini-batches, we have to ensure that different batches are balanced and they can be used to represent the whole dataset. Typical sizes are 10, 100 or 1000 data points. **Epoch** is one complete pass through over the entire training dataset.

2 Wether forecasting using real data from SMHI

The objective of this assignment is implement a kernel method to predict hourly temperatures for a specific location and date in Sweden. The target location is **Örnsköldsviks Flygplats** (63.4119° N, 18.9797° E) with a real temperature of -7.7° and the target date is **January 31, 2012**.

2.1 Data Processing

The prediction model utilizes a database of weather station readings provided by SMHI. By applying a Gaussian kernel smoother, the model estimates the temperature by calculating a weighted average of historical observations. The weights are determined by the proximity of the observations to the target prediction in terms of three dimensions: physical distance, date difference, and time of day.

The data from `stations.csv` and `temps50k.csv` are merged to associate the temperature readings with the geographical coordinates. To ensure realistic forecasting, we apply a causal filter: for any specific forecast hour t , only observations recorded prior to t are included in the calculation.

2.2 The Kernel Function

A Gaussian kernel is chosen to define the similarity between the target point and the historical observations. The kernel function is defined as:

$$k(d, h) = \exp\left(-\frac{d^2}{2h^2}\right) \quad (1)$$

Where:

- d is the distance between the target and the observation (in space, date, or time).
- h is the bandwidth (smoothing parameter) that controls the width of the kernel.

2.3 Distance Metrics

Three distinct distances were calculated for each observation:

1. **Spatial Distance** (d_{space}): Calculated using the Haversine formula to account for the Earth’s curvature.
2. **Date Distance** (d_{date}): The absolute difference in days between the observation date and the forecast date.
3. **Time Distance** (d_{time}): The difference in hours between the observation time and the forecast time. This metric accounts for the cyclical nature of the day (e.g., the distance between 23:00 and 01:00 is calculated as 2 hours).

2.4 Product of Kernels

The implementation utilizes the **Product** of three separate kernels to calculate the final weight w_i for each observation i . This approach requires that an observation be close in *all* dimensions (space AND date AND time) to have a significant influence on the prediction.

$$w_i = k(d_{space}, h_{space}) \times k(d_{date}, h_{date}) \times k(d_{time}, h_{time}) \quad (2)$$

The predicted temperature $\hat{T}$ is the weighted average of the historical temperatures T_i :

$$\hat{T} = \frac{\sum_i w_i \cdot T_i}{\sum_i w_i} \quad (3)$$

2.5 Bandwidths

The following smoothing coefficients are selected to balance local accuracy with sufficient data density:

- $h_{distance} = 20,000$ meters (20 km): Ensures only geographically close stations contribute significantly.
- $h_{date} = 10$ days: Captures seasonal trends while smoothing out daily volatility.
- $h_{time} = 4$ hours: Accounts for the diurnal cycle, utilizing neighboring hours to inform the prediction.

2.6 Results

The forecast is generated for 2-hour intervals from 04:00 to 24:00 (00:00). The resulting plot for our chosen location is shown as below.

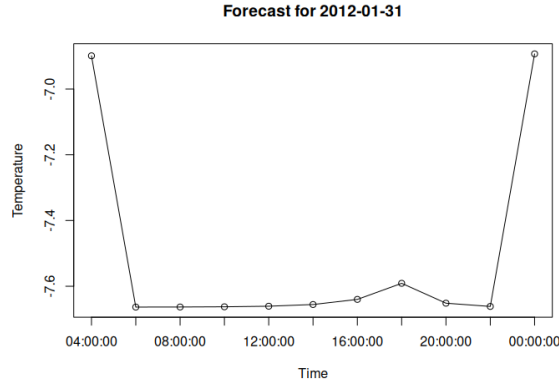


Figure 1: the forecasted temperature for Örnskoldsvik Flygplats 2012-01-31 at 6 am

As it is shown in (Figure 1) the forecasted temperature (around -7.2°C) is very close to the actual temperature (-7.7°C).

2.7 Conclusion

2.7.1 Summation vs. Multiplication of Kernels

The lab instructions proposed comparing the sum of kernels against the multiplication of kernels. This report utilizes the **Multiplication** method ($w = k_1 \cdot k_2 \cdot k_3$).

- **Multiplication (Product Kernel):** This treats the probability as a joint density. It is highly selective; an observation must be relevant in terms of location, season, *and* time of day to contribute. This effectively eliminates the noise from distant stations or different seasons.
- **Summation (Additive Kernel):** A summation approach ($w = k_1 + k_2 + k_3$) would allow a point that is very close in space but far in time (e.g., summer) to still have influence. The multiplication method is strictly more rigorous for finding "similar" weather contexts.

The kernel smoothing method successfully generated a temperature profile for Örnköldsvik. By using a product of Gaussian kernels with bandwidths of 20km, 10 days, and 4 hours, the model effectively isolated relevant historical data to construct a plausible temperature trend for the target date.

3 Support Vector Machines

Filter name	Trained on	Tested against	Generalization error
Filter 0	Training	Validation	0.165
Filter 1	Training	Test	0.1672909
Filter 2	Training, Validation	Test	0.1498127
Filter 3	Training, Validation, Test	Test	0.01373283

3.1 Which filter do you return to the user?

We return filter 2 to the user. This is the one filter that is only trained non-test data and which has the lowest generalization error.

3.2 What is the estimate of the generalization error of the filter returned to the user?

The generalization error is the error generated by the second filter = 0.1498127. It is trained on both the training and validation data sets and validated against the test data set, which contains.

By method of elimination:

- **Filter 0** is trained on the training data set and validated against the validation data set. However, since the validation data set is used to tune the hyperparameter C , it is data which has been seen by the model before, and thus cannot be used to validate the model's performance on completely new data.

- **Filter 1** is trained on the training data set and tested against test. This would work, however we can also use the validation data set in training so filter 1 is not optimal.
- **Filter 3** is trained on the entire data set, including the test partition. However, this means that when the model is tested against the test data set, that will be data which it has seen before. This means that it cannot be used to calculate a generalization error.

Filter 2 is trained on both the training and validation data sets, and tested against the test set. Thus, the generalization error is properly calculated, and we use the most amount of data possible for training (both training and validation sets). Thus, this is the most accurate generalization error of the options.

3.3 Implement a linear combination for filter 3

```
kfun <- rbfdot(sigma = 0.05)

k<-NULL
for(i in 1:10){
  k2<-0
  for(j in 1:length(sv)){
    k2<-k2 + co[j] * kfun(as.numeric(spam[sv[j], -58]), as.numeric(spam[i, -58]))
  }
  k2 <- k2 + inte
  k<-c(k, k2)
}
k
predict(filter3,spam[1:10,-58], type = "decision")

===== RESULTS =====
[1,] -1.0702965
[2,]  1.0003450
[3,]  0.9995908
[4,] -0.9999648
[5,] -0.9995379
[6,]  1.0000612
[7,] -0.8585873
[8,] -0.9997047
[9,]  0.9998209
[10,] -1.0000973
```

Here, for the first ten data points, we iterate over the support vectors of filter 3 and compute the linear combination according to this function:

$$f(\mathbf{x}) = \sum_{j=1}^m \alpha_j y_j \exp\left(-\frac{\|\mathbf{x}_j - \mathbf{x}\|^2}{2\sigma^2}\right) + b$$

where m is the amount of support vectors, σ is the bandwidth and b is the intercept. A positive numerical result means a positive result, as in it is spam.

4 Neural Networks

4.1 Training a Neural Network

Before training can begin, we need to generate initial random weights for the neural networks. To compute the exact number of weights, we utilize the following formula: $(inputNeurons + 1) * hiddenUnits + (hiddenUnits + 1) * outputNeurons$. As a result, we get 31 total weights and we generate initial weights with this code snippet:

```
winit <- runif(totalWeights, min = -1, max = 1)
```

Figure 2 demonstrates the plot of training, test and prediction data. We can see that predictions(red) generally follows the actual data(blue). In addition, training and testing MSEs are utilized to evaluate a neural network. They are **0.00116151943719272** and **0.0138867742451697** respectively. Based on this, we can confidently say that the model is decent.

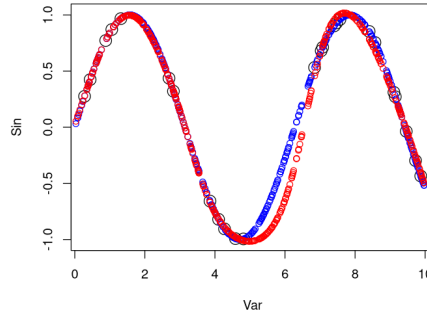


Figure 2: Plot of training, test and prediction data

4.2 Training a NN with various activation functions

Figure 3 and Table 1 show the summary of a neural network utilizing activation function $h_1(x) = x$. The data indicates that this activation function is not suitable for hidden layers, because it fires up every neuron in the network and the network collapses into 1 layer. This network is no different from a linear regression and the result speaks for itself.

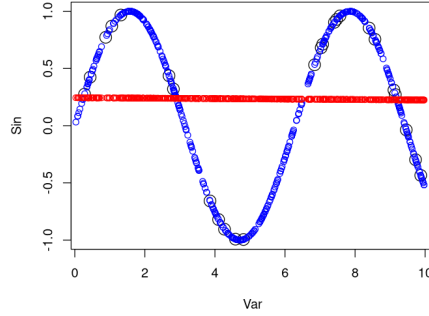


Figure 3: Plot of training, test and prediction data ($h_1(x) = x$)

Training MSE	Testing MSE
0.4389878327683	0.44673077089838

Table 1: Summary of MSEs ($h_1(x) = x$)

Figure 4 and Table 2 show the summary of a neural network using the activation function $h_2(x) = \max\{0, x\}$. The prediction curve follows the testing curve in the beginning; however, when **Var** reaches approximately **4**, the prediction curve takes a sharp incline and becomes linear. This is a classic symptom of the "dying ReLu" problem. During training, too many neurons have died (most likely due to the high learning rate) and the network is now unable to predict sin when $Var \geq 4$. To fix this, we can either increase the number of neurons in the hidden layer or utilize **Leaky ReLu**. The jagged prediction lines can be explained by having too few neurons in the hidden layer for ReLu to model curvature.

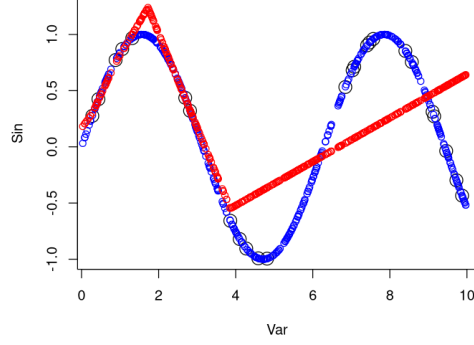


Figure 4: Plot of training, test and prediction data ($h_3(x) = \ln(1 + \exp x)$)

Training MSE	Testing MSE
0.26413710322059	0.198349586353266

Table 2: Summary of MSEs ($h_3(x) = \ln(1 + \exp x)$)

Figure 5 and Table 3 show the summary of a neural network using the activation function $h_3(x) = \ln(1 + \exp x)$. **Softplus** is essentially a smoother version of ReLu. It can be used to model curvatures and is especially useful in smaller networks, where ReLu causes neurons to die. Based on results, it is the best among all other networks with the lowest **Test MSE**.

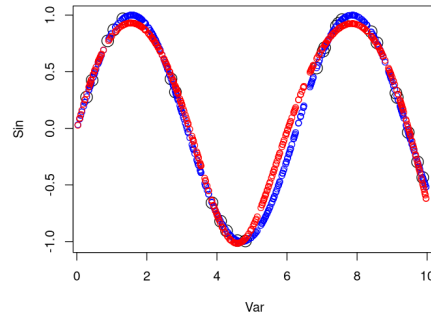


Figure 5: Plot of training, test and prediction data ($h_3(x) = \max\{0, x\}$)

Training MSE	Testing MSE
0.00160935399902943	0.00925561307565867

Table 3: Summary of MSEs ($h_3(x) = \max\{0, x\}$)

4.3 Predicting unseen data with NN

Figure 6 illustrates the prediction curve. The initial model has been trained on the interval $[0, 10]$, but now it tries to predict $\sin$ in the interval $[0, 50]$. When $Var > 10$ the prediction begins to suffer, because the neural network has never seen values in this range. As Var increases, neurons begin to output zeros and ones in a more consistent manner, until the "pattern" stabilizes and the values converge around -7 .

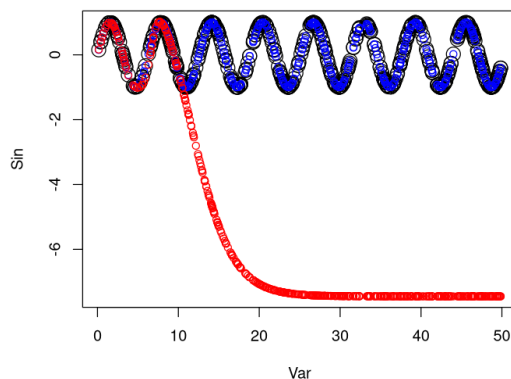


Figure 6

4.4 Calculating the convergence value

Predicting $\sin$ on values above 50 yields bad results. Because neural network has never been trained on these values, the higher input values causes the sigmoid function to get saturated. As a consequence, all output values are extremely close to 0 or 1, so the network is no longer sensitive to changes to x . Convergence value itself is -7.45105063629257 .

4.5 Predicting training data

Figure 7 illustrates how neural network predicts the training data. The plot clearly, shows that results are absolutely bad. This happens, because while $y = \sin(x)$ is a function, the inverse is not. The neural network tries to learn weights for this function, but if we look at $\sin$ wave, we can see that it repeats

itself every 2π radians. For example for input 0, sin can be 0, π or 2π . This causes the loss function to become "confused", because it is getting fed the contradictory data, thus making it impossible to train the neural network.

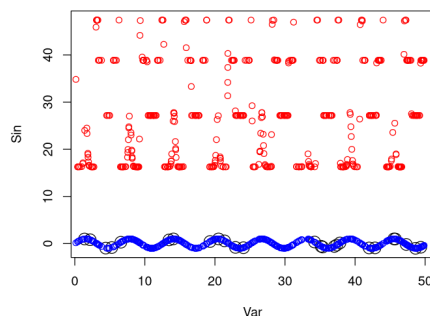


Figure 7

A Appendix

A.1 Assignment 2 code

```
set.seed(1234567890)
library(geosphere)

# 1. Read and merge data
stations <- read.csv("Assignment1/Data/stations.csv", fileEncoding = "latin1")
temps    <- read.csv("Assignment1/Data/temps50k.csv")
st <- merge(stations, temps, by = "station_number")

#Kernel function
k_gauss <- function(d, h) {
  exp(-(d^2) / (2 * h^2))
}

#Bandwidths
h_distance <- 20000 # m
h_date    <- 10     # days
h_time    <- 4      # hours

# 4. Prediction point and datep
a <- 63.4119 # Örnköldsvíks Flygplats
b <- 18.9797 # Örnköldsvíks Flygplats
```

```

date <- "2012-01-31"
forecast_date <- as.Date(date)
st$date <- as.Date(st$date)

st$time <- as.character(st$time)

# Combined datetime for filtering out future observations
st$datetime <- as.POSIXct(paste(st$date, st$time), tz = "UTC")

for_times <- c("04:00:00", "06:00:00", "08:00:00", "10:00:00",
               "12:00:00", "14:00:00", "16:00:00", "18:00:00",
               "20:00:00", "22:00:00", "24:00:00")
for_times[for_times == "24:00:00"] <- "00:00:00"
temp <- vector(length = length(for_times))

d_space <- distHaversine(
  cbind(st$longitude, st$latitude),
  c(b, a)
)

d_date <- abs(as.numeric(st$date - forecast_date))

# print(length(d_space)); print(nrow(st))
# summary(d_space)
# summary(d_date)

#Loop over forecast times
for (j in seq_along(for_times)) {

  this_time <- for_times[j]
  forecast_dt <- as.POSIXct(paste(date, this_time), tz = "UTC")

  idx_past <- which(st$datetime <= forecast_dt)

  if (length(idx_past) == 0) {
    temp[j] <- NA
    next
  }

  # Subset
  d_space_past <- d_space[idx_past]
  d_date_past <- d_date[idx_past]
  temps_past <- st$air_temperature[idx_past]
  times_past <- st$time[idx_past]

  obs_times <- as.POSIXct(paste("1970-01-01", times_past), tz = "UTC")

```

```

fcast_time_dummy <- as.POSIXct(paste("1970-01-01", this_time), tz = "UTC")

diff_linear <- abs(as.numeric(difftime(obs_times, fcast_time_dummy, units = "hours")))
d_time <- pmin(diff_linear, 24 - diff_linear) # Handle 23:00 vs 01:00

# Kernels
k_space <- k_gauss(d_space_past, h_distance)
k_date <- k_gauss(d_date_past, h_date)
k_time <- k_gauss(d_time, h_time)

w <- k_space * k_date * k_time
#w <- k_space + k_date + k_time

if (sum(w) == 0) {
  temp[j] <- NA
} else {
  temp[j] <- sum(w * temps_past) / sum(w)
}
}

# Plot results
plot(temp, type = "o", xaxt = "n", xlab = "Time", ylab = "Temperature", main = paste("Forecast", for_times))
axis(1, at = 1:length(for_times), labels = for_times)

```

A.2 Assignment 3 code

```

# Lab 3 block 1 of 732A99/TDDE01/732A68 Machine Learning
# Author: jose.m.pena@liu.se
# Made for teaching purposes
library(kernlab)
set.seed(1234567890)

data(spam)
foo <- sample(nrow(spam))
spam <- spam[foo,]
tr <- spam[1:3000, ]
va <- spam[3001:3800, ]
trva <- spam[1:3800, ]
te <- spam[3801:4601, ]

#We find the best C by using cross-validation. We start with C = 0.3 and increment
# by 0.3 up to 5.0.
by <- 0.3
err_va <- NULL
for(i in seq(by,5,by)){
  filter <- ksvm(type~.,data=tr,kernel="rbfdot",kpar=list(sigma=0.05),C=i,scaled=FALSE)

```

```

mailtype <- predict(filter,va[,-58])
t <- table(mailtype,va[,58])
err_va <-c(err_va,(t[1,2]+t[2,1])/sum(t))
}

#Tests training against validation.
#This does not work because the validation data set has already been used when determining C
filter0 <- svm(type~.,data=tr,kernel="rbfdot",kpar=list(sigma=0.05),C=which.min(err_va)*by,
mailtype <- predict(filter0,va[,-58])
t <- table(mailtype,va[,58])
err0 <- (t[1,2]+t[2,1])/sum(t)
err0

#Tests training against test.
#Training against test should not result in anything bad, since we are not reusing any data
#This should generate the generalization error.
filter1 <- ksvm(type~.,data=tr,kernel="rbfdot",kpar=list(sigma=0.05),C=which.min(err_va)*by,
mailtype <- predict(filter1,te[,-58])
t <- table(mailtype,te[,58])
err1 <- (t[1,2]+t[2,1])/sum(t)
err1

#Tests training+validation against test. Seems the most logical.
#This is the generalization error: It trains the model as much as it can on already used data
#tests it on previously unseen data. This is the generalization error, which is 0.1498127
#Since we have used the validation data set for tuning the hyperparameter C, it is not a good
#it as additional training data since it has been used in tuning.
filter2 <- ksvm(type~.,data=trva,kernel="rbfdot",kpar=list(sigma=0.05),C=which.min(err_va)*by,
mailtype <- predict(filter2,te[,-58])
t <- table(mailtype,te[,58])
err2 <- (t[1,2]+t[2,1])/sum(t)
err2

# Tests the entire dataset, including test, against test? Unsure why.
filter3 <- ksvm(type~.,data=spam,kernel="rbfdot",kpar=list(sigma=0.05),C=which.min(err_va)*by,
mailtype <- predict(filter3,te[,-58])
t <- table(mailtype,te[,58])
err3 <- (t[1,2]+t[2,1])/sum(t)
err3

sv<-alphaindex(filter3)[[1]]
co<-coef(filter3)[[1]]
inte<- - b(filter3)

kfun <- rbfdot(sigma = 0.05)

```

```

k<-NULL
for(i in 1:10){
  k2<-0
  for(j in 1:length(sv)){
    k2<-k2 + co[j] * kfun(as.numeric(spam[sv[j], -58]), as.numeric(spam[i, -58]))
  }
  k2 <- k2 + inte
  k<-c(k, k2)
}
k
predict(filter3,spam[1:10,-58], type = "decision")

```

A.3 Assignment 4 code

```

set.seed(1234567890)
library(neuralnet)

SamplePoints <- function(n, min, max) {
  set.seed(1234567890)
  Var <- runif(n, min, max)
  return(data.frame(Var, Sin = sin(Var)))
}

NN <- function(trainData, testData, activationFn = "logistic", summarize = TRUE) {
  set.seed(1234567890)
  inputNeurons = 1
  hiddenNeurons = 10
  outputNeurons = 1
  totalWeights = (inputNeurons + 1) * hiddenNeurons + (hiddenNeurons + 1) * outputNeurons

  winit <- runif(totalWeights, min = -1, max = 1)
  nn <- neuralnet(
    Sin ~ Var,
    data = tr,
    hidden = c(hiddenNeurons),
    startweights = winit,
    act.fct = activationFn,
    linear.output = TRUE
  )

  if (summarize) {
    SummarizeNN(nn, trainData, testData)
  }

  return(nn)
}

```

```

SummarizeNN <- function(nn, trainData, testData) {
  set.seed(1234567890)

  preds <- predict(nn, testData)
  yLimits = range(c(testData$Sin, preds))
  plot(trainData, cex = 2, ylim = yLimits)
  points(testData, col = "blue", cex = 1)
  points(testData[, 1],
         predict(nn, testData),
         col = "red",
         cex = 1)

  trainMSE <- mean((trainData$Sin - predict(nn, trainData))^2)
  testMSE <- mean((testData$Sin - preds)^2)
  print(paste("Train MSE:", trainMSE))
  print(paste("Test MSE:", testMSE))
}

# ===== Task 1 =====
mydata <- SamplePoints(500, 0, 10)
tr <- mydata[1:25, ]
te <- mydata[26:500, ]

nn <- NN(trainData = tr, testData = te)
# =====

# ===== Task 2 =====
nn_ <- NN(trainData = tr, testData = te, activationFn = function(x) x)

nn_ <- NN(trainData = tr, testData = te, activationFn = function(x) ifelse(x > 0, x, 0))

nn_ <- NN(trainData = tr, testData = te, activationFn = function(x) log(1 + exp(x)))
# =====

# ===== Task 3 =====
mydata <- SamplePoints(500, 0, 50)
SummarizeNN(nn, mydata, mydata)
# Lab assistant Lisa said, that Sin axis looks a bit weird and minimum value on it should be 0
# =====

# ===== Task 4 =====
layer1Matrix <- nn$weights[[1]][[1]]

biasWeights <- layer1Matrix[1,]
inputWeights <- layer1Matrix[-1, , drop = FALSE]
outputWeights <- nn$weights[[1]][[2]]

```

```

conValue <- outputWeights[1]

for (i in 1:length(inputWeights)) {
  z <- (inputWeights[i] * 50) + biasWeights[i]
  sigma <- 1 / (1 + exp(-z))
  conValue <- conValue + (sigma * outputWeights[i + 1])
}

print(paste("Convergence value:", conValue))
# =====

# ===== Task 5 =====
mydata <- SamplePoints(500, 0, 50)
tr <- mydata[1:25,]
te <- mydata[26:500,]

set.seed(1234567890)
inputNeurons = 1
hiddenNeurons = 10
outputNeurons = 1
totalWeights = (inputNeurons + 1) * hiddenNeurons + (hiddenNeurons + 1) * outputNeurons

winit <- runif(totalWeights, min = -1, max = 1)
nn <- neuralnet(
  Var ~ Sin,
  data = tr,
  hidden = c(hiddenNeurons),
  startweights = winit,
  threshold = 0.1,
  linear.output = TRUE
)
SummarizeNN(nn, tr, te)
# =====

```